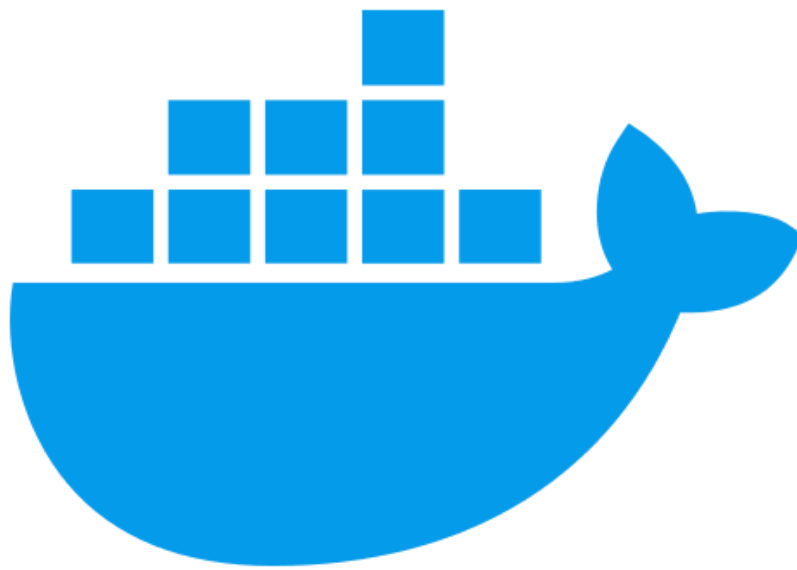

SWAP P5: BENCHMARKING

ÁLVARO GARCÍA ZAFRA



TIEMPO

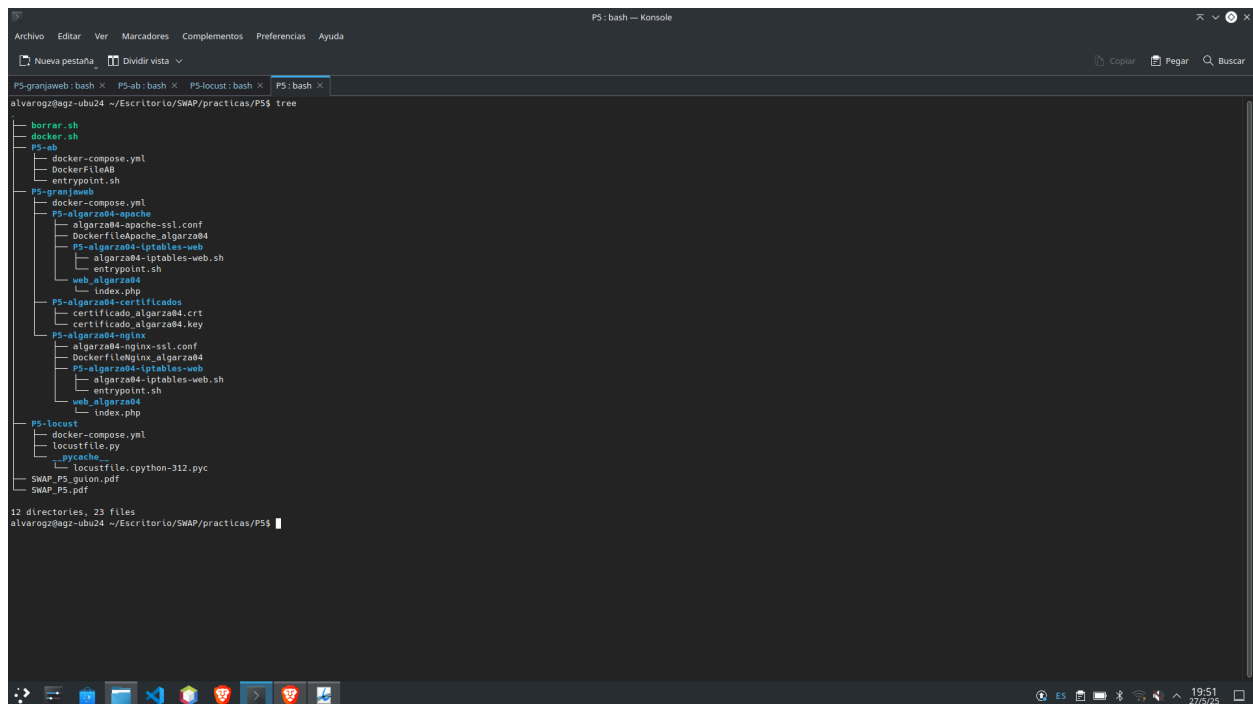
BÁSICAS

Tareas Básicas - B1. Configuración del entorno de benchmarking

El primer paso de esta práctica ha sido copiar los archivos de la práctica anterior y adaptarlos a las necesidades de esta práctica, es decir adaptar los nombres, dejar solo la parte de las tareas básicas y añadirlas al directorio P5-granjaweb.

Para finalizar las necesidades de esta parte he creado el directorio P5-ab, donde se incluirá todo lo relacionado con Apache Benchmark, y el directorio P5-locust, donde se incluirá los archivos y configuración de locust.

A continuación se puede ver como se ha quedado el esquema de la práctica tras realizar las tareas avanzadas:



```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P5$ tree
.
├── borrar.sh
├── docker.sh
├── P5-ab
│   ├── docker-compose.yml
│   ├── DockerfileAB
│   └── entrypoint.sh
├── P5-granjaweb
│   ├── docker-compose.yml
│   ├── P5-algarza04-apache
│   │   ├── algarza04-apache-ssl.conf
│   │   ├── DockerfileApache_algarza04
│   │   ├── P5-algarza04-iptables-web
│   │   │   ├── algarza04-iptables-web.sh
│   │   │   └── entrypoint.sh
│   │   └── web_algarza04
│   │       └── index.php
│   ├── P5-algarza04-certificados
│   │   ├── certificado_algarza04.crt
│   │   └── certificado_algarza04.key
│   ├── P5-algarza04-nginx
│   │   ├── algarza04-nginx-ssl.conf
│   │   ├── DockerfileNginx_algarza04
│   │   ├── P5-algarza04-iptables-web
│   │   │   ├── algarza04-iptables-web.sh
│   │   │   └── entrypoint.sh
│   │   └── web_algarza04
│   │       └── index.php
├── P5-locust
│   ├── docker-compose.yml
│   ├── locustfile.py
│   ├── pycache
│   ├── locustfile.cpython-312.pyc
├── SWAP_P5_guiion.pdf
└── SWAP_P5.pdf

12 directories, 23 files
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P5$
```

En esta captura se pueden apreciar 2 script, [docker.sh](#) y [borrar.sh](#), la función de estos es levantar todo el sistema, es decir, realizar los docker-compose de los 3 directorios, mientras que el segundo sirve para detener su ejecución y eliminar todos docker e imágenes existentes en el dispositivo.

Tareas Básicas - B2. Implementación con Apache Benchmark

Una vez creados todos los directorios podemos crear los archivos de configuración necesarios. En esta tarea crearemos los de Apache Benchmark, en concreto su DockerFile y su docker-compose:

```
P5-ab > DockerFileAB > ...
1  # El contenedor se crea a partir de la última imagen Debian
2  FROM debian:latest
3
4  # Indica que no se solicitará interacción con el usuario durante la instalación de paquetes
5  ENV DEBIAN_FRONTEND=noninteractive
6
7  # Actualiza los paquetes, instala el conjunto de herramientas de apache, que contendrá ab
8  # elimina los archivos de la lista de paquetes
9  RUN apt-get update && apt-get install -y apache2-utils && \
10     rm -rf /var/lib/apt/lists/*
11
12 # Copia el archivo entrypoint al contenedor
13 COPY entrypoint.sh /entrypoint.sh
14
15 # Da permisos de ejecución al script entrypoint.sh
16 RUN chmod +x /entrypoint.sh
17
18 # Define dicho script como punto de entrada principal del contenedor
19 ENTRYPOINT ["/entrypoint.sh"]
```

```
P5-ab > $ entrypoint.sh
1  #!/bin/bash
2
3  # Ejecuta el comando de Apache Benchmark (ab) para realizar pruebas de carga
4  # Mandará 10000 peticiones con una concurrencia de 100 tanto a HTTPS como a HTTP
5  ab -n 10000 -c 100 https://192.168.10.50:443/
6  ab -n 10000 -c 100 http://192.168.10.50:80/
7
8  # Luego, ejecuta el comando principal del contenedor
9  exec "$@"
```

Estos dos archivos que estamos viendo son el DockerFile y el script entrypoint que le pasaré como punto de entrada. El funcionamiento de estos archivos es crear un contenedor Apache Benchmark cuya utilidad será realizar pruebas de carga, para esto mandará una serie de peticiones a los puertos HTTPS y HTTP del balanceador nginx.

Por último podemos ver el contenido del docker-compose que se encargará de la creación del contenedor, y su imagen, a partir del DockerFile anterior:

```
P5-ab > docker-compose.yml > ...
1  # establecemos la red web como externa
2  networks:
3    red_web:
4      external: true
5
6  ▶ Run All Services
7  services:
8    # inicialización del contenedor apache-benchmark-P5:
9    # -es creado a partir del DockerFileAB
10   # -se le asigna su nombre e imagen
11   # -se asigna la red web con su ip estática correspondiente
12   ▶ Run Service
13   apache-benchmark-P5:
14     build:
15       context: .
16       dockerfile: DockerFileAB
17     image: algarza04-ab-image:p5
18     container_name: apache_benchmark-P5
19     networks:
20       red_web:
21         ipv4_address: 192.168.10.60
```

Para finalizar con este apartado voy a comprobar si el servicio se levanta correctamente:

```
alvarog@egz-ubu24 ~/Escritorio/SWAP/practicas/P5/P5-ab$ docker-compose up -d
Building apache-benchmark-P5
[+] Building 1.1s (9/9) FINISHED
=> [internal] load build definition from DockerFileAB
=> => transferring dockerfile: 741B
=> [internal] load metadata for docker.io/library/debian:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/debian:latest@sha256:bd73076dc2cd9c88f48b5b358328f24f2a4289811bd73787c031e20db9f97123
=> [internal] load build context
=> => transferring context: 35B
=> CACHED [2/4] RUN apt-get update && apt-get install -y apache2-utils && rm -rf /var/lib/apt/lists/*
=> CACHED [3/4] COPY entrypoint.sh /entrypoint.sh
=> CACHED [4/4] RUN chmod +x /entrypoint.sh
=> exporting to image
=> => exporting layers
=> => writing image sha256:fdb070c38b0abac9f572dca2391e896d5d587f1d96c43e7ec449e8d630390da5
=> => naming to docker.io/library/algarza04-ab-image:p5
WARNING: Image for service apache-benchmark-P5 was built because it did not already exist. To rebuild this image you must use 'docker-compose build' or 'docker-compose up --build'.
Creating apache_benchmark-P5 ...
alvarog@egz-ubu24 ~/Escritorio/SWAP/practicas/P5/P5-ab$ docker ps | grep apache_benchmark-P5
21a9b47be281   algarza04-ab-image:p5   "/entrypoint.sh"      5 seconds ago    Up 4 seconds
alvarog@egz-ubu24 ~/Escritorio/SWAP/practicas/P5/P5-ab$
```

Como se puede apreciar en la captura anterior se está levantando correctamente y podemos ver que al ejecutarlo se queda levantado hasta que realice la ejecución que esperamos.

Tareas Básicas - B3. Implementación con Locust

En la tarea anterior implementé todo lo necesario para el contenedor AB y en este haré lo mismo para el de Locust. Para esto crearé un docker-compose y un script con lo que debe hacer:

```
P5-locust > docker-compose.yml > ...
1  # establecemos la red web como externa
2  networks:
3    red_web:
4      external: true
5
6  ▷ Run All Services
7  services:
8    # Ejecuta Locust en modo master, en el puerto 8089
9    # Monta el locustfile.py que hemos implementado en el dir /mnt/locust
10   # Por último se asigna una dirección IP estática de la red web
11   ▷ Run Service
12   master-algarza04:
13     image: locustio/locust
14     ports:
15       - "8089:8089"
16     volumes:
17       - ./mnt/locust
18     command: -f /mnt/locust/locustfile.py --master
19     networks:
20       red_web:
21         ipv4_address: 192.168.10.70
22
23   #Este servicio lanza 6 réplicas de workers locust, conectados a master
24   #Todos ellos utilizan el mismo script .py que el servicio master
25   ▷ Run Service
26   worker-algarza04:
27     image: locustio/locust
28     volumes:
29       - ./mnt/locust
30     command: -f /mnt/locust/locustfile.py --worker --master-host master-algarza04
31     depends_on:
32       - master-algarza04
33     deploy:
34       replicas: 6
35     networks:
36       - red_web
```

```
P5-locust > locustfile.py > ...
1  # Importa las siguientes librerías de Locust
2  # -httpsuser servirá para definir un usuario que realizará las peticiones
3  # -taskset servirá para definir un conjunto de tareas que realizará el usuario
4  # -task servirá para definir una tarea que realizará el usuario
5  # -between servirá para definir el tiempo de espera entre tareas
6  from locust import HttpUser, TaskSet, task, between
7
8  # Simula un usuario recibiendo la página index.php y desactiva la verificación del certificado SSL
9  class P5_algarza04(TaskSet):
10     @task
11     def load_index(self):
12         self.client.get("/index.php", verify=False)
13
14  # Se definen dos tipos de usuarios:
15  # -P5_usuarios_https: usuario que realiza peticiones a través de HTTPS
16  # -P5_usuarios_http: usuario que realiza peticiones a través de HTTP
17  # Ambos usuarios tienen un tiempo de espera entre 1 y 5 segundos entre tareas e indica las tareas que debe realizar
18  class P5_usuarios_https(HttpUser):
19     host = "https://192.168.10.50:443"
20     tasks = [P5_algarza04]
21     wait_time = between(1, 5)
22
23  class P5_usuarios_http(HttpUser):
24     host = "http://192.168.10.50:80"
25     tasks = [P5_algarza04]
26     wait_time = between(1, 5)
```

El primer archivo es el docker-compose, el cuál se encargará de crear el entorno Locust que servirá para realizar las pruebas de carga. Este creará un master que se encargará de coordinar a los 6 workers y de recoger y mostrar las estadísticas que estos le envíen. Por otro lado se encuentra el script locustfile, el cuál define las tareas que deben realizar los usuarios (solicitar y simular el uso del index.php), además define dos tipos de usuarios:

- Usuarios http: ejecutarán la tarea sin cifrado, por el puerto 80.
- Usuarios https: ejecutarán la tarea utilizando cifrado, puerto 443.

Para finalizar este apartado voy a ejecutar su docker compose y así poder comprobar que está todo bien implementado:

```
alvarogz@gz-ubu24 ~/Escritorio/SWAP/practicas/PS/PS-locust$ docker-compose up -d
Pulling master-algarza04 (locustio/locust)...
latest: Pulling from locustio/locust
61320b01ae5e: Already exists
be1274d3cce0: Pull complete
8091c928062: Pull complete
7a1cb080221: Pull complete
b0e8451853fb: Pull complete
4b41692184aa: Pull complete
4a1091a0f3cc: Pull complete
da0855c570ef: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:7ebd75954608568b854fd3e016c21acc0acfb04dc07e0ed53aea8dcf6a1b58413
Status: Downloaded newer image for locustio/locust:latest
Creating p5-locust_master-algarza04_1 ... done
Creating p5-locust_worker-algarza04_1 ... done
Creating p5-locust_worker-algarza04_2 ... done
Creating p5-locust_worker-algarza04_3 ... done
Creating p5-locust_worker-algarza04_4 ... done
Creating p5-locust_worker-algarza04_5 ... done
Creating p5-locust_worker-algarza04_6 ... done
alvarogz@gz-ubu24 ~/Escritorio/SWAP/practicas/PS/PS-locust$ docker ps | grep locust
80927b04915 locustio/locust *locust -f /mnt/locu... 15 seconds ago Up 14 seconds 5557/tcp, 8089/tcp p5-locust_worker-algarza04_1
c065a0b3844d locustio/locust *locust -f /mnt/locu... 15 seconds ago Up 14 seconds 5557/tcp, 8089/tcp p5-locust_worker-algarza04_6
8350d475133e locustio/locust *locust -f /mnt/locu... 15 seconds ago Up 14 seconds 5557/tcp, 8089/tcp p5-locust_worker-algarza04_3
5185cc2844d2 locustio/locust *locust -f /mnt/locu... 15 seconds ago Up 14 seconds 5557/tcp, 8089/tcp p5-locust_worker-algarza04_5
83571313b5d4d locustio/locust *locust -f /mnt/locu... 15 seconds ago Up 14 seconds 5557/tcp, 8089/tcp p5-locust_worker-algarza04_2
863409d42d07 locustio/locust *locust -f /mnt/locu... 15 seconds ago Up 14 seconds 5557/tcp, 8089/tcp p5-locust_worker-algarza04_4
47da85eedacd locustio/locust *locust -f /mnt/locu... 15 seconds ago Up 14 seconds 5557/tcp, 0.0.0.0:8089->8089/tcp, [::]:8089->8089/tcp p5-locust_master-algarza04_1
alvarogz@gz-ubu24 ~/Escritorio/SWAP/practicas/PS/PS-locust$
```

De esta manera he podido asegurar que estos servicios están correctamente implementados y no tienen a priori ningún error de implementación.

Tareas Básicas - B4. Ejecución de pruebas de carga

Una vez tenemos todo lo anterior implementado correctamente vamos a pasar a ejecutar todos los escenarios, para esto seguiremos el siguiente orden: granjaweb->ab->locust.

A continuación se puede ver su despliegue y su posterior comprobación:

```

P5-granjaweb: bash × P5-ab: bash × P5-locust: bash × P5: bash ×
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P5/P5-granjaweb$ docker-compose up -d
Creating network "red_web" with the default driver
Creating network "red_servicios" with the default driver
Building web1
[+] Building 1.2s (14/14) FINISHED
=> [internal] load build definition from DockerfileApache_algarza04
=> => transferring dockerfile: 1.92kB
=> [internal] load metadata for docker.io/library/debian:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/9] FROM docker.io/library/debian:latest@sha256:bd73076dc2cd9c88f48b5b358328f24f2a4289811bd73787c031e20db9f97123
=> [internal] load build context
=> => transferring context: 3.23kB
=> CACHED [2/9] RUN apt-get update && apt-get install -y apache2 php libapache2-mod-php iputils-ping iptables && rm -rf /var/lib/apt/lists/*
=> CACHED [3/9] COPY P5-algarza04-apache/web_algarza04/index.php /var/www/html/
=> CACHED [4/9] COPY P5-algarza04-certificados/certificado_algarza04.crt /etc/apache2/ssl/certificado_algarza04.crt
=> CACHED [5/9] COPY P5-algarza04-certificados/certificado_algarza04.key /etc/apache2/ssl/certificado_algarza04.key
=> CACHED [6/9] COPY P5-algarza04-apache/algarza04-apache-ssl.conf /etc/apache2/sites-available/algarza04-apache-ssl.conf
=> CACHED [7/9] COPY P5-algarza04-apache/P5-algarza04-iptables-web/entrypoint.sh /entrypoint.sh
=> CACHED [8/9] COPY P5-algarza04-apache/P5-algarza04-iptables-web/algarza04-iptables-web.sh /algarza04-iptables-web.sh
=> CACHED [9/9] RUN a2enmod ssl && a2ensite default-ssl && chmod 600 /etc/apache2/ssl/certificado_algarza04.crt && chmod +x /entrypoint.sh /algarza04-iptables-web.sh
=> exporting to image
=> => exporting layers
=> => writing image sha256:6c06aba634ff677b65dbd564cea6872df4fb49fed3b1071f5ff93f1777c6c891
=> => naming to docker.io/library/algarza04-apache-image:p5

1 warning found (use docker --debug to expand):
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 41)
WARNING: Image for service web1 was built because it did not already exist. To rebuild this image you must use 'docker-compose build' or 'docker-compose up --build'.
Building balancerador-nginx-ssl
[+] Building 0.6s (10/10) FINISHED
=> [internal] load build definition from DockerfileNginx_algarza04
=> => transferring dockerfile: 746B
=> [internal] load metadata for docker.io/library/nginx:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/nginx:latest@sha256:fb39280b7b9eba5727c884a3c7810002e69e8f961cc373b89c92f14961d903a0
=> [internal] load build context
=> => transferring context: 3.10kB
=> CACHED [2/5] RUN mkdir -p /etc/nginx/ssl
=> CACHED [3/5] COPY P5-algarza04-certificados/certificado_algarza04.crt /etc/nginx/ssl/certificado_algarza04.crt
=> CACHED [4/5] COPY P5-algarza04-certificados/certificado_algarza04.key /etc/nginx/ssl/certificado_algarza04.key
=> CACHED [5/5] COPY P5-algarza04-nginx/algarza04-nginx-ssl.conf /etc/nginx/nginx.conf
=> exporting to image
=> => exporting layers
=> => writing image sha256:bdb8bc2eeb8db3bae4d0b7dccc7e867061dff7def01e61b3f2a02f87153176
=> => naming to docker.io/library/algarza04-nginx-image:p5

1 warning found (use docker --debug to expand):
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 19)
WARNING: Image for service balancerador-nginx-ssl was built because it did not already exist. To rebuild this image you must use 'docker-compose build' or 'docker-compose up --build'.
Creating web3 ... done
Creating web2 ... done
Creating web5 ... done
Creating web1 ... done
Creating web4 ... done
Creating web7 ... done
Creating web6 ... done
Creating web8 ... done
Creating balancerador-nginx ... done

alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P5/P5-ab$ docker-compose up -d
Building apache-benchmark-P5
[+] Building 0.3s (9/9) FINISHED
=> [internal] load build definition from DockerFileAB
=> => transferring dockerfile: 438B
=> [internal] load metadata for docker.io/library/debian:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/debian:latest@sha256:bd73076dc2cd9c88f48b5b358328f24f2a4289811bd73787c031e20db9f97123
=> [internal] load build context
=> => transferring context: 35B
=> CACHED [2/4] RUN apt-get update && apt-get install -y apache2-utils && rm -rf /var/lib/apt/lists/*
=> CACHED [3/4] COPY entrypoint.sh /entrypoint.sh
=> CACHED [4/4] RUN chmod +x /entrypoint.sh
=> exporting to image
=> => exporting layers
=> => writing image sha256:f8a4aa562c870875b2adb563552c918c89f9f4bbf1bf890106bd3db3ccbf3f6f
=> => naming to docker.io/library/algarza04-ab-image:p5
WARNING: Image for service apache-benchmark-P5 was built because it did not already exist. To rebuild this image you must use 'docker-compose build' or 'docker-compose up --build'.
Creating apache_benchmark-P5 ... done

alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P5/P5-locust$ docker-compose up -d
Pulling master-algarza04 (locustio/locust:...)
latest: Pulling from locustio/locust
61320b01ae5e: Already exists
be1274d3cce0: Pull complete
0091c9200d62: Pull complete
7a1cb0808221: Pull complete
b6e8451853fb: Pull complete
4b41692184aa: Pull complete
44109160f3cc: Pull complete
da0855c57c0f: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:7ebd75954600568b854fd3e016c21acc0acf04dc67e0ed53aea8dcf6a1b58413
Status: Downloaded newer image for locustio/locust:latest
Creating p5-locust_master-algarza04_1 ... done
Creating p5-locust_worker-algarza04_1 ... done
Creating p5-locust_worker-algarza04_2 ... done
Creating p5-locust_worker-algarza04_3 ... done
Creating p5-locust_worker-algarza04_4 ... done
Creating p5-locust_worker-algarza04_5 ... done
Creating p5-locust_worker-algarza04_6 ... done

```

```

alvarogz@qz-ubu24 ~/Escritorio/5WAP/practicas/P5$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED    STATUS    PORTS                               NAMES
3e3ffe769e7   locustio/locust   "locust -f /mnt/locu..." 3 seconds ago    Up 2 seconds    5557/tcp, 8089/tcp              p5-locust_worker-algarza04_5
19bbee64789b   locustio/locust   "locust -f /mnt/locu..." 3 seconds ago    Up 2 seconds    5557/tcp, 8089/tcp              p5-locust_worker-algarza04_2
3a5c7833d8e6   locustio/locust   "locust -f /mnt/locu..." 3 seconds ago    Up 2 seconds    5557/tcp, 8089/tcp              p5-locust_worker-algarza04_3
a0fb7c18f6a7   locustio/locust   "locust -f /mnt/locu..." 3 seconds ago    Up 2 seconds    5557/tcp, 8089/tcp              p5-locust_worker-algarza04_1
4912593383c6   locustio/locust   "locust -f /mnt/locu..." 3 seconds ago    Up 2 seconds    5557/tcp, 8089/tcp              p5-locust_worker-algarza04_4
10a8bce821eb   locustio/locust   "locust -f /mnt/locu..." 3 seconds ago    Up 2 seconds    5557/tcp, 8089/tcp              p5-locust_worker-algarza04_6
433e9ca7a77a   locustio/locust   "locust -f /mnt/locu..." 3 seconds ago    Up 2 seconds    5557/tcp, 0.0.0.0:8089->8089/tcp, [::]:8089->8089/tcp  p5-locust_master-algarza04_1
166287a2f1fa   algarza04-ab-image:p5   "/entrypoint.sh"          15 seconds ago   Up 14 seconds                                apache.benchmark-p5
a0b591d63861   algarza04-nginx-image:p5   "/docker-entrypoint..." 19 seconds ago   Up 18 seconds                                balancador-nginx
a04654391c44   algarza04-apache-image:p5   "/entrypoint.sh /bin..." 20 seconds ago   Up 19 seconds                                web6
382418985d33   algarza04-apache-image:p5   "/entrypoint.sh /bin..." 20 seconds ago   Up 19 seconds                                web7
6a7a565a3d0f   algarza04-apache-image:p5   "/entrypoint.sh /bin..." 20 seconds ago   Up 19 seconds                                web4
1904c39a8386   algarza04-apache-image:p5   "/entrypoint.sh /bin..." 20 seconds ago   Up 19 seconds                                web5
60eea808bb85   algarza04-apache-image:p5   "/entrypoint.sh /bin..." 20 seconds ago   Up 19 seconds                                web1
7d4dc4943a66   algarza04-apache-image:p5   "/entrypoint.sh /bin..." 20 seconds ago   Up 19 seconds                                web5
527f802f83f   algarza04-apache-image:p5   "/entrypoint.sh /bin..." 20 seconds ago   Up 19 seconds                                web2
b05bancea072   algarza04-apache-image:p5   "/entrypoint.sh /bin..." 20 seconds ago   Up 19 seconds                                web3
alvarogz@qz-ubu24 ~/Escritorio/5WAP/practicas/P5$

```

Una vez hemos realizado esto el contenedor AB habrá mandado 10000 solicitudes a http y el mismo número a https, como podemos ver en la siguiente comprobación:

```

Active connections: 1
server accepts handled requests
20077 20077 20002
Reading: 0 Writing: 1 Waiting: 0

```

Lo que nos indica que ha recibido todas las solicitudes.

Tareas Básicas - B5. Análisis de resultados

Para finalizar las tareas básicas vamos a analizar los datos obtenidos con las pruebas realizadas con AB y con Locust.

En primer lugar voy a analizar los de AB, para esto he accedido a los logs de dicho contenedor, como podemos ver en las siguientes capturas. En ellas podemos observar que se han realizado 10000 peticiones (con un nivel de concurrencia de 100) a cada uno de los tipos de clientes establecidos, http y https, además nos está indicando que su tasa de fallos es 0, por lo que no está ocurriendo ningún error en estos.

Otro aspecto que se puede apreciar es el tiempo que tarda cada una de las solicitudes), las solicitudes https tienen mayor tiempo de respuesta que las http (172 ms frente a los 17ms de http). Esto es debido a que las solicitudes https deben trabajar con los certificados, los cuales tienen que cifrar/descifrar, dando lugar a una mayor latencia y uso de recursos, lo que se traduce a un mayor tiempo de respuesta (mientras que https necesita 17,2 segundos para realizar todas las solicitudes http lo realiza en 1,2 segundos), terminando https con una tasa de respuesta 10 veces inferior a la de http (150,4Kb/s frente a 1510,3Kb/s).

```

Benchmarking 192.168.10.50 (be patient)
Completed 1000 requests
Completed 2000 requests
Completed 3000 requests
Completed 4000 requests
Completed 5000 requests
Completed 6000 requests
Completed 7000 requests
Completed 8000 requests
Completed 9000 requests
Completed 10000 requests
Finished 10000 requests

Server Software:      nginx/1.27.5
Server Hostname:      192.168.10.50
Server Port:          443
SSL/TLS Protocol:     TLSv1.3,TLS_AES_256_GCM_SHA384,2048,256
Server Temp Key:      X25519 253 bits

Document Path:        /
Document Length:      86 bytes

Concurrency Level:     100
Time taken for tests:   17.268 seconds
Complete requests:     10000
Failed requests:        0
Total transferred:     2660000 bytes
HTML transferred:      860000 bytes
Requests per second:   579.10 [#/sec] (mean)
Time per request:      172.680 [ms] (mean)
Time per request:      1.727 [ms] (mean, across all concurrent requests)
Transfer rate:         150.43 [Kbytes/sec] received

Connection Times (ms)
      min    mean[+/-sd] median   max
Connect:    6   104  40.2    98   360
Processing: 11    68  35.7    63   411
Waiting:    1    37  29.2    31   397
Total:      18   172  50.9   161   570

Percentage of the requests served within a certain time (ms)
 50%    161
 66%    174
 75%    180
 80%    187
 90%    210
 95%    271
 98%    357
 99%    382
100%    570 (longest request)

```

```

Benchmarking 192.168.10.50 (be patient)
Completed 1000 requests
Completed 2000 requests
Completed 3000 requests
Completed 4000 requests
Completed 5000 requests
Completed 6000 requests
Completed 7000 requests
Completed 8000 requests
Completed 9000 requests
Completed 10000 requests
Finished 10000 requests

Server Software:      nginx/1.27.5
Server Hostname:      192.168.10.50
Server Port:          80

Document Path:        /
Document Length:      86 bytes

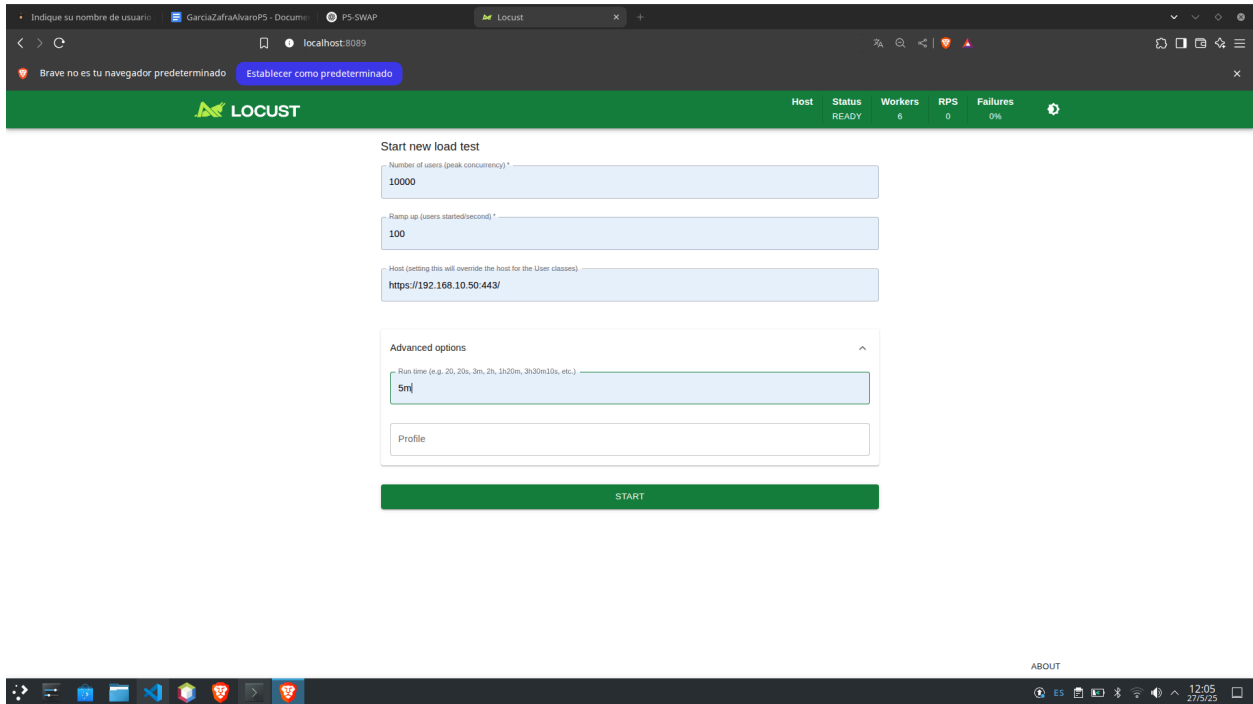
Concurrency Level:     100
Time taken for tests:   1.720 seconds
Complete requests:     10000
Failed requests:        0
Total transferred:     2660000 bytes
HTML transferred:      860000 bytes
Requests per second:   5814.32 [#/sec] (mean)
Time per request:      17.199 [ms] (mean)
Time per request:      0.172 [ms] (mean, across all concurrent requests)
Transfer rate:         1510.36 [Kbytes/sec] received

Connection Times (ms)
      min    mean[+/-sd] median   max
Connect:    0     6   0.9     6     9
Processing:  2    11   1.3    11    21
Waiting:    1     9   1.0     9    17
Total:      6    17   1.3    17    25

Percentage of the requests served within a certain time (ms)
 50%    17
 66%    18
 75%    18
 80%    18
 90%    19
 95%    19
 98%    20
 99%    20
100%    25 (longest request)

```

Una vez hemos analizado las pruebas realizadas con AB pasaremos al análisis de las realizadas con locust. Para este análisis he realizado 2 pruebas de 10000 peticiones, con un nivel de concurrencia de 100, mandando una al host con el puerto 443 y otra al host con el puerto 80.



HTTPS

LOCUST

Host
https://192.168.10.50:443/

Status
SPAWNING

Users
1216

Workers
6

RPS
32.56

Failures
0%

EDIT

STOP

RESET

STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

! LOGS

WORKERS

LOCUST CLOUD

Worker	State	# users	CPU usage	Memory usage
0e12e7a7a5d4_6e6c921ddc7e44ff8bb9846cd8ca5199	spawning	200	67.7	622.77 MB
265390f62ab0_a68066052b0c48f7b2bb838277c0b3f1	spawning	200	85.1	651.96 MB
6532dd47232c_0a6ee641f0ce4ebc81142bb35d998a7a	spawning	167	76.8	629.94 MB
c2b15bb0ba46_f4db5228da02447b855b643dd37c262f	running	233	59.1	618.02 MB
df716618f51e_37d746d83ed94da1ba85178efc295186	spawning	216	20.5	604.94 MB
e268cf2fe97b_524f3003021a4ee2a3f2f679313229d7	running	200	21.7	612.85 MB

LOCUST

Host
https://192.168.10.50:443/

Status
SPAWNING

Users
3023

Workers
6

RPS
161

Failures
0%

EDIT

STOP

RESET

STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

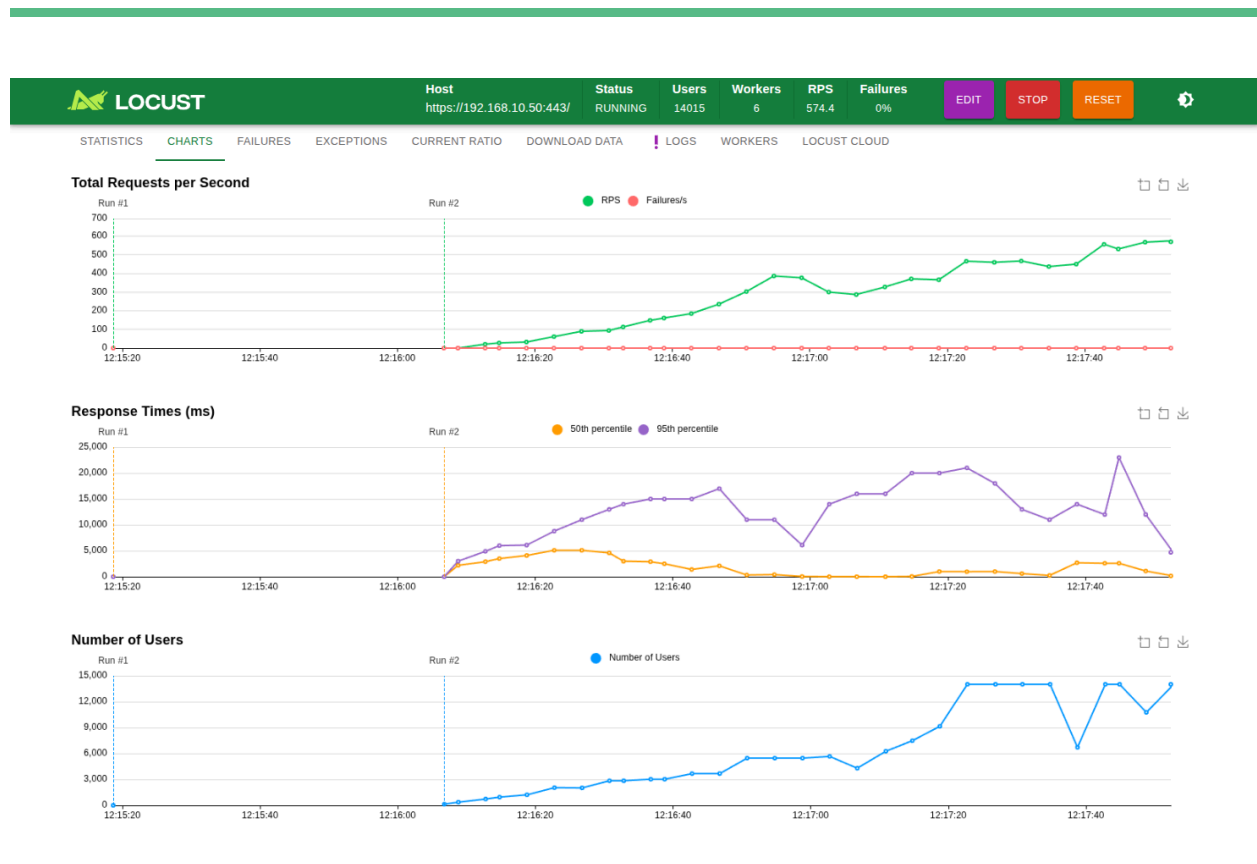
DOWNLOAD DATA

! LOGS

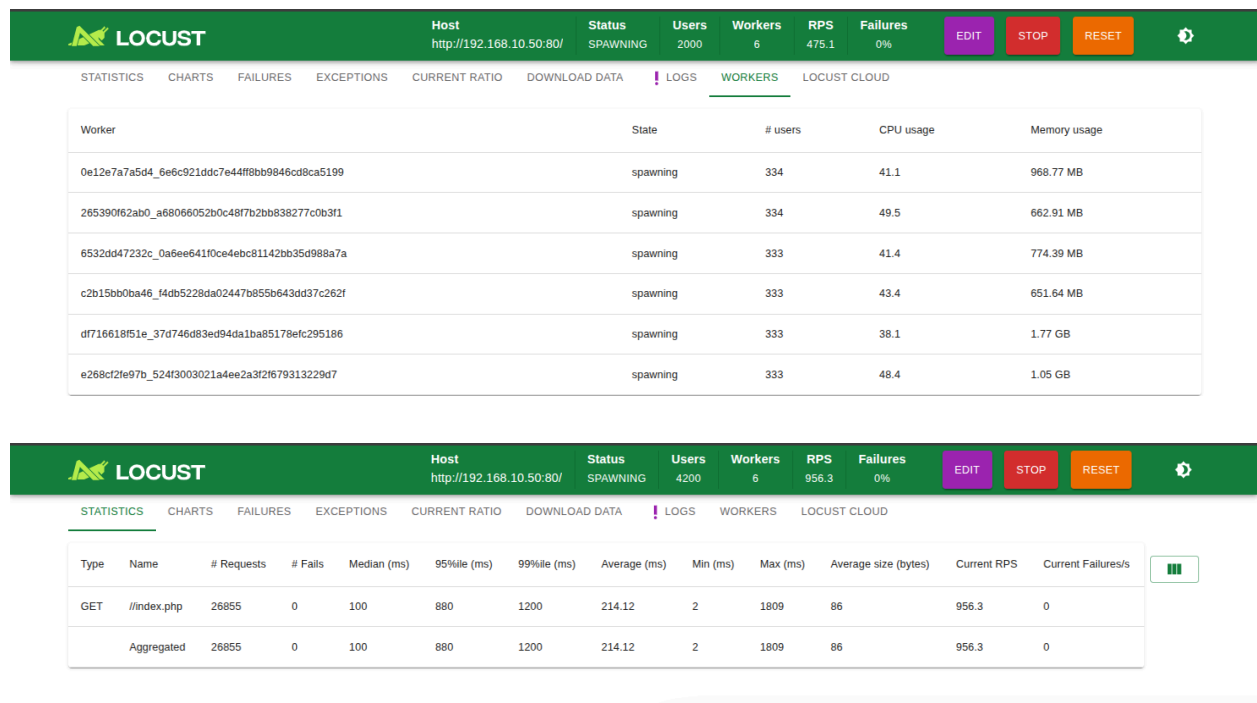
WORKERS

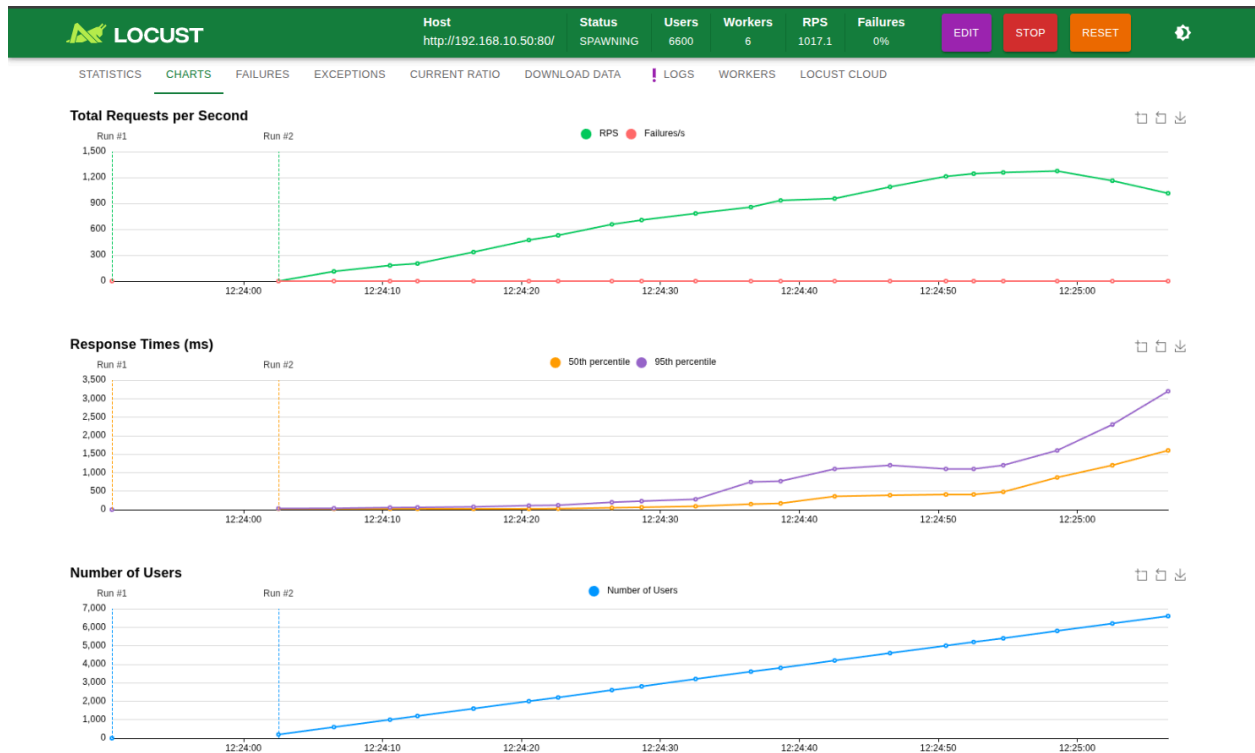
LOCUST CLOUD

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/index.php	3265	0	3300	13000	17000	4342.36	2	18852	86	161	0
	Aggregated	3265	0	3300	13000	17000	4342.36	2	18852	86	161	0



HTTP





En las pruebas realizadas con locust podemos observar que al igual que con AB me ha dado una tasa de errores de 0, lo cuál significa que las pruebas se han llevado a cabo de manera correcta.

Al observar la salida de cada uno podemos observar una serie de diferencias. Por lo general los workers al recibir solicitudes https tienen un mayor porcentaje en el uso de CPU, lo cual será debido a la necesidad de trabajar con certificados. Al igual que ha pasado con AB he podido comprobar que la velocidad de procesamiento de http es muy superior a la de https (956 RPS de http frente a 131 de https, lo cuál es debido a lo mismo que en el caso anterior, trabajar con certificados), lo que se traduce en un tiempo de respuesta muy superior en el caso de http.

Por último voy a comparar sus dos gráficas, en ellas se aprecia que http es mucho más lineal, aumentando el número de RPS, tiempo de respuesta y su número de usuarios, mientras que https está en constante subida y bajada en estos datos y con peores resultados en ellos, lo cuál es un ejemplo gráfico de los aspectos que he comentado anteriormente.

Finalmente al comparar ambos análisis podemos observar que los resultados han sido muy similares entre sí y coinciden con lo que se esperaba de ellos.

AVANZADAS

Tareas Avanzadas - A1. Desarrollo avanzado en Locustfile.py

En este apartado voy a incorporar tareas que realizarán operaciones típicas del CMS. Para esto he modificado mi locustfile añadiendo autenticación de usuarios, también estuve intentando añadir confirmaciones de post, pero por algún motivo esto me daba 100% de error, por lo que decidí quitarlo.

Locustfile con los cambios realizados:

```
# Importa las siguientes librerías de Locust
# -HttpUser servirá para definir un usuario que realizará las peticiones
# -TaskSet servirá para definir un conjunto de tareas que realizará el usuario
# -task servirá para definir una tarea que realizará el usuario
# -between servirá para definir el tiempo de espera entre tareas
from locust import HttpUser, TaskSet, task, between

# Simula un usuario realizando tareas en el CMS y desactiva la verificación del certificado SSL
from locust import HttpUser, TaskSet, task, between

class P5_algarza04(TaskSet):
    def on_start(self):
        self.login()

    def login(self):
        # Hacer login en index.php?action=login
        response = self.client.post(
            "index.php?action=login",
            json={"username": "testuser", "password": "testpass"},
            verify=False
        )
        if response.status_code == 200:
            token = response.json().get("token")
            self.headers = {"Authorization": f"Bearer {token}"}
        else:
            self.headers = {}

    @task
    def load_index(self):
        # Cargar la página principal
        self.client.get("index.php", verify=False)

# Se definen dos tipos de usuarios:
# -P5_usuarios_https: usuario que realiza peticiones a través de HTTPS
# -P5_usuarios_http: usuario que realiza peticiones a través de HTTP
# Ambos usuarios tienen un tiempo de espera entre 1 y 5 segundos entre tareas e indica las tareas que debe realizar
class P5_usuarios_https(HttpUser):
    host = "https://192.168.10.50:443"
    tasks = [P5_algarza04]
    wait_time = between(1, 5)

class P5_usuarios_http(HttpUser):
    host = "http://192.168.10.50:80"
    tasks = [P5_algarza04]
    wait_time = between(1, 5)
```

Index con los cambios realizados:

```

<?php
header('Content-Type: application/json');

// Mostrar IP del servidor si se accede sin acción específica
if ($_SERVER['REQUEST_METHOD'] === 'GET' && empty($_GET['action'])) {
    $server_ip = $_SERVER['SERVER_ADDR'];
    echo '<h1>SWAP - "Álvaro García Zafra"</h1><br>';
    echo "Dirección IP del servidor: $server_ip \n";
    exit;
}

// Acción esperada
$action = $_GET['action'] ?? '';

if ($_SERVER['REQUEST_METHOD'] === 'POST' && $action === 'login') {
    // Leer cuerpo JSON
    $input = json_decode(file_get_contents("php://input"), true);

    // Login simple
    $username = $input['username'] ?? '';
    $password = $input['password'] ?? '';

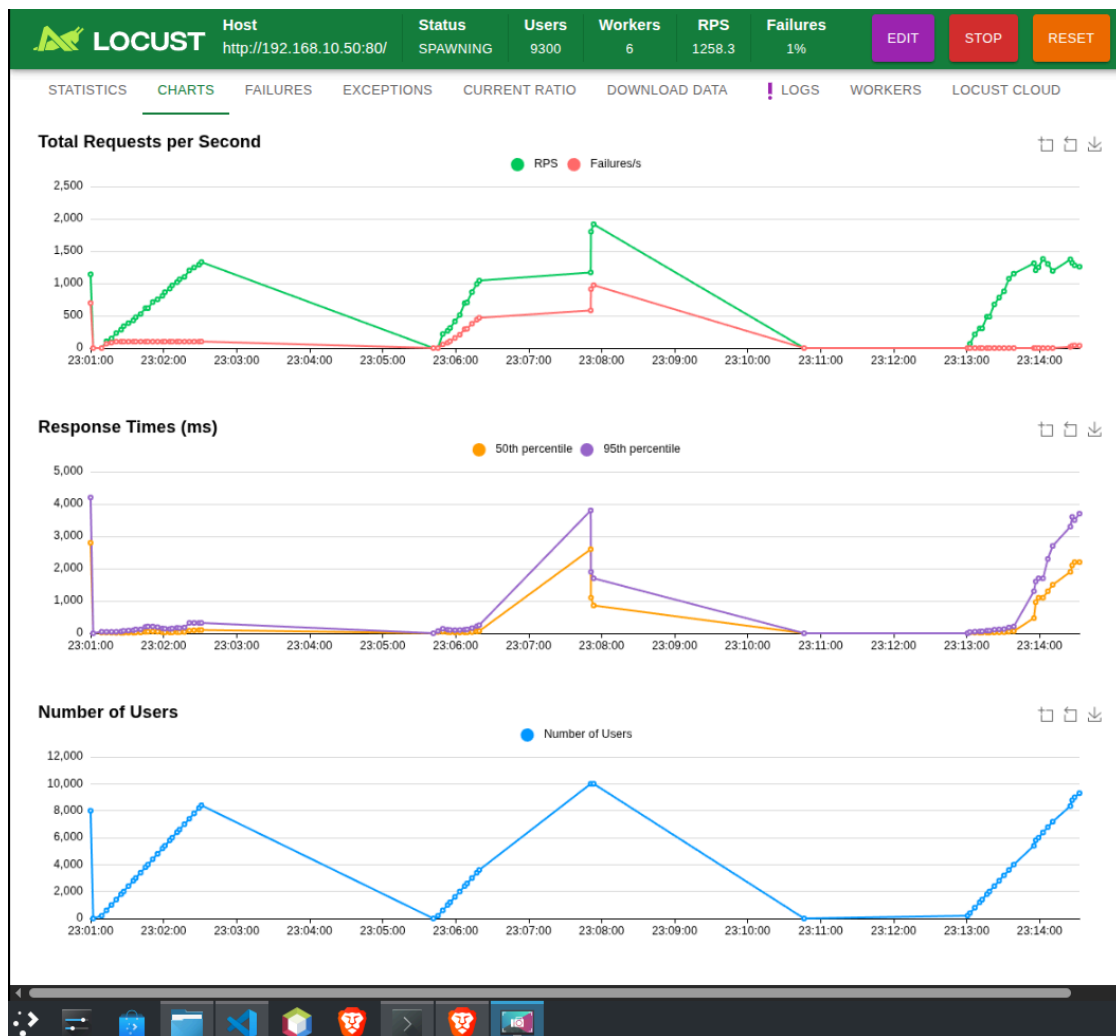
    if ($username === 'testuser' && $password === 'testpass') {
        $token = bin2hex(random_bytes(16)); // Token de prueba
        file_put_contents("token.txt", $token); // Guardar el token
        echo json_encode(["token" => $token]);
    } else {
        http_response_code(401);
        echo json_encode(["error" => "Credenciales inválidas"]);
    }
} else {
    http_response_code(400);
    echo json_encode(["error" => "Petición inválida"]);
}
?>

```

A continuación veremos un ejemplo de la ejecución en locust, para el cuál configure Locust con 10000 peticiones con un nivel de concurrencia de 100:

 LOCUST Host: http://192.168.10.50:80/ Status: SPAWNING Users: 3600 Workers: 6 RPS: 1072.1 Failures: 0% EDIT STOP RESET												
STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA ! LOGS WORKERS LOCUST CLOUD												
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/index.php	22722	0	22	130	190	38.22	2	593	85	970.4	0
POST	/index.php?action=login	3595	0	79	200	270	92.16	12	309	44	101.7	0
Aggregated		26317	0	30	150	210	45.59	2	593	79.4	1072.1	0

LOCUST		Host	Status	Users	Workers	RPS	Failures			
		http://192.168.10.50:80/	SPAWNING	6383	6	1378.2	0%	EDIT	STOP	RESET
STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS WORKERS LOCUST CLOUD										
Worker			State	# users	CPU usage	Memory usage				
04b1b049ed77_acce663585f34c8eb068decda899bb64			running	1067	85.1	93.7 MB				
184786ee8864_bbebf537acc24da29cbf75655f8a2342			running	1067	88.4	92.91 MB				
263f6ead091_10c52591126141cc89e26c9a90fe6e4			spawning	1050	83.8	96.48 MB				
7b0dcf718480_884bbe82e60d4edcb458f19e9f5acad4			running	1067	87.2	93.11 MB				
8f8e2d56d403_96062238632340a9a35638a7ce1ea928			running	1066	85.9	93.26 MB				
90204177205e_b90457d349ba439c873c1db8e5237550			running	1066	85.8	94.19 MB				



Como podemos apreciar en la salida no se ha producido una notable sobrecarga, esta ha sido simplemente del 1%, teniendo el index.php una media de 38 peticiones por ms, mientras que login tiene una media de 92 por ms.

ANÁLISIS PROPUESTA IA

Enlace al chat: <https://chatgpt.com/share/683cc396-284c-800d-ba15-9a3546888c46>

Resumen y opinión de su uso

En este último apartado voy a hablar sobre el uso de IA a lo largo de esta práctica.

En esta práctica el uso de la inteligencia artificial me ha sido de gran ayuda, pero no me ha resuelto algunos problemas de una manera rápida y sencilla, lo cuál se puede deber al poco control que tenía en estas nuevas tecnologías que he tenido que usar y el poco tiempo que he tenido para su aprendizaje.

A lo largo de las tareas básicas no lo necesité en gran medida, simplemente para saber algunos términos de uso de docker, como el uso de command para ejecutar lo de AB y para configurar correctamente el segundo puerto del locustfile.

En cambio, para realizar las avanzadas lo necesite para la mayor parte del ejercicio resuelto. En primer lugar no sabía lo que tenía que hacer y una vez lo entendí intenté ayudarme con su uso pero me costó mucho, el ejercicio me estaba dando muchos problemas y finalmente tuve que hacer su implementación de manera más sencilla.